

Aulas 6 e 7 – Estruturas de Dados Lineares: Listas, Filas e Pilhas e Estruturas estáticas e dinâmicas

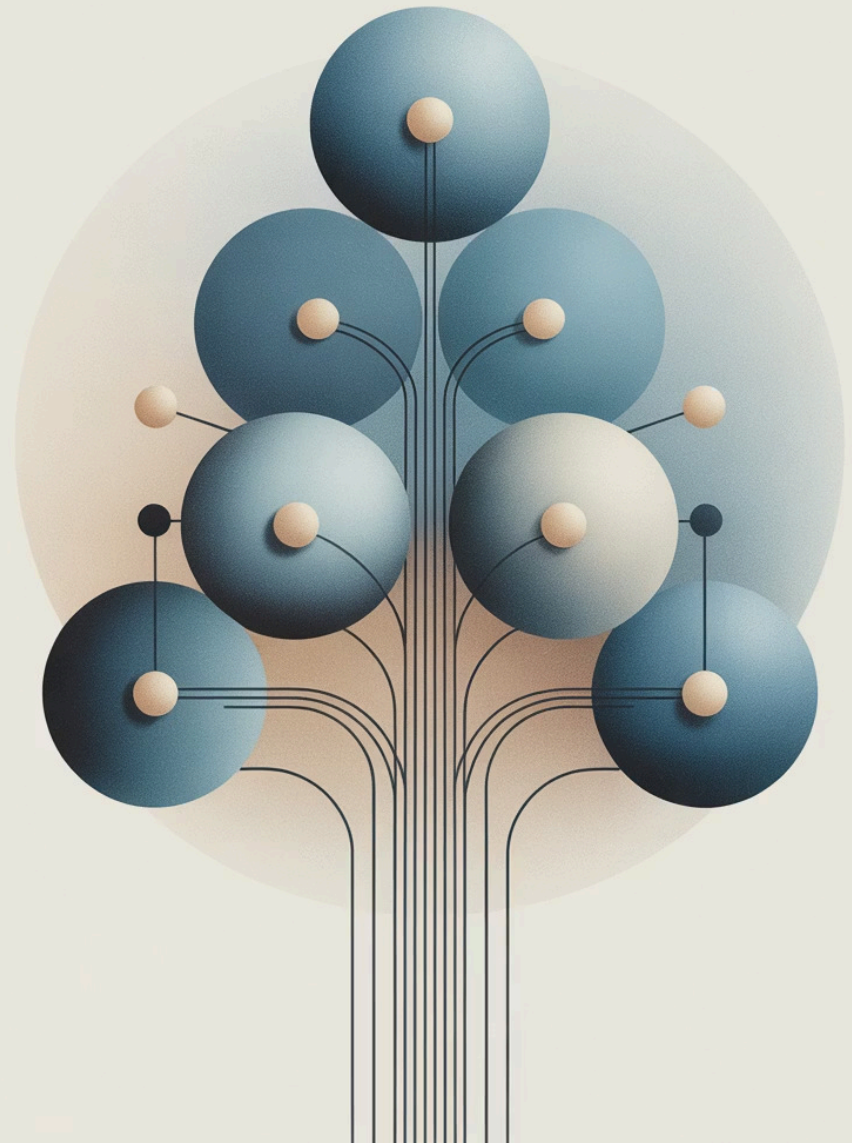
Universidade Federal de Goiás

Instituto de Matemática e Estatística e Instituto de Informática (INF/UFG)

Curso: Matemática Aplicada e Computacional

Disciplina: INF0286 – Algoritmos e Estruturas de Dados I

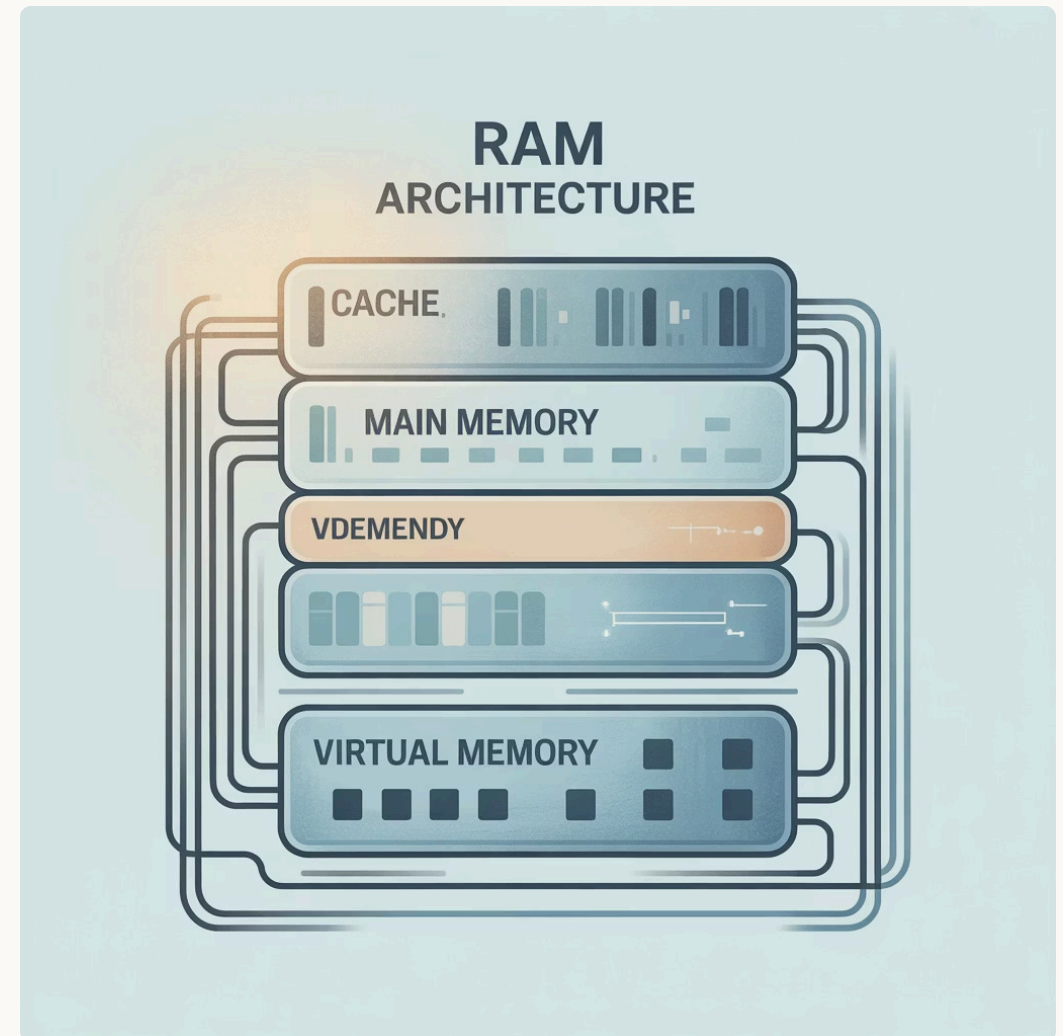
Professora: Dra. Renata Dutra Braga



O que são Estruturas de Dados?

As estruturas de dados são formas organizadas de armazenar e manipular informações na memória do computador. Elas representam a base fundamental para o desenvolvimento de algoritmos eficientes e sistemas computacionais robustos.

Todo trabalho com estruturas de dados envolve dois aspectos fundamentais que determinam sua aplicabilidade e eficiência em diferentes contextos computacionais.

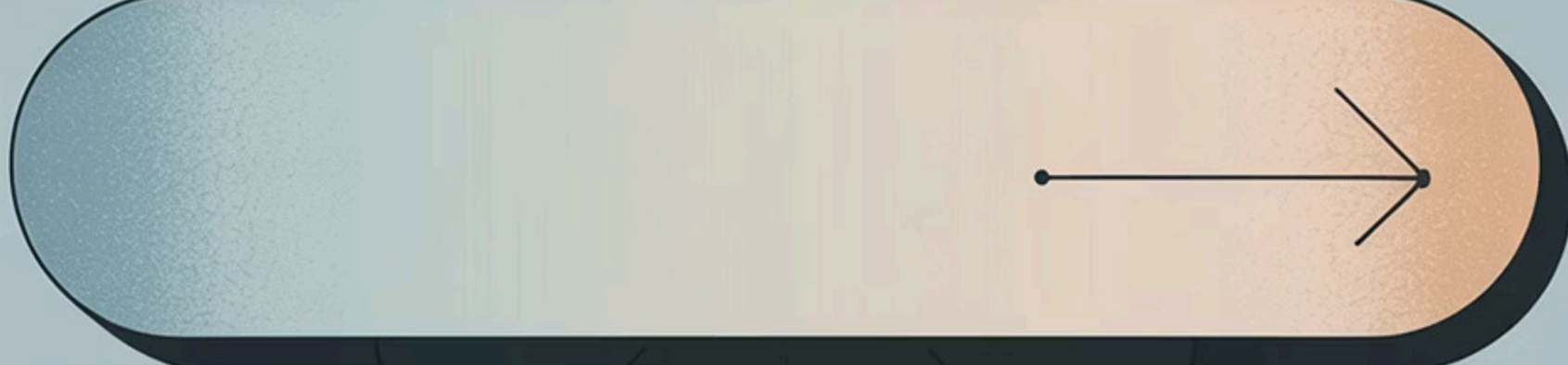


Aspecto Estrutural

Define como os dados estão fisicamente organizados na memória, estabelecendo as relações espaciais entre os elementos armazenados.

Aspecto Funcional

Determina quais operações podem ser realizadas sobre os dados, incluindo inserção, remoção, busca e modificação de elementos.



Estruturas de Dados Lineares

As estruturas lineares organizam elementos de forma sequencial, onde cada elemento possui uma relação de ordem bem definida com seus vizinhos. Esta organização sequencial facilita a compreensão e implementação, tornando-as fundamentais no estudo de algoritmos.

01

Lista

Elementos organizados em sequência, onde cada item possui uma posição específica e pode ser acessado diretamente através de índices.

02

Pilha (Stack)

Estrutura com acesso restrito ao topo, seguindo o princípio LIFO (Last In, First Out) para inserção e remoção de elementos.

03

Fila (Queue)

Estrutura onde elementos entram por uma extremidade e saem pela outra, seguindo o princípio FIFO (First In, First Out).

Listas Lineares: Fundamentos

Uma lista linear é uma coleção de elementos $x_1, x_2, \dots, x_n$ organizados em uma sequência ordenada, onde cada elemento mantém uma relação de precedência com seus vizinhos.



Primeiro Elemento (x_1)

Elemento inicial da lista que não possui antecessor, servindo como ponto de entrada para percorrer toda a estrutura sequencialmente.



Último Elemento (x_n)

Elemento final da lista que não possui sucessor, marcando o término da sequência e servindo como limite para operações de travessia.



Relações de Ordem

Cada elemento intermediário possui exatamente um antecessor e um sucessor, criando uma cadeia contínua de relacionamentos ordenados.

Operações Principais

- Acesso direto a elementos através de índices
- Inserção e remoção em posições arbitrárias
- Ordenação e combinação de múltiplas listas

Implementações de Listas

As listas podem ser implementadas de diferentes formas, cada uma com características específicas que influenciam diretamente na eficiência das operações e no uso da memória.

Implementação Estática

Utiliza vetores com tamanho fixo definido em tempo de compilação. Oferece acesso $O(1)$ a qualquer posição, mas apresenta limitações para operações de inserção e remoção no meio da estrutura.

- Acesso direto por índice
- Memória contígua
- Tamanho fixo predefinido

Implementação Dinâmica

Emprega ponteiros para conectar elementos, permitindo crescimento e redução durante a execução. Oferece flexibilidade total, mas requer gerenciamento cuidadoso da memória.

- Tamanho variável
- Uso eficiente de memória
- Inserção/remoção flexível

 **Variações Especiais:** Listas simples, circulares e duplamente encadeadas oferecem diferentes níveis de funcionalidade e eficiência para casos específicos de uso.

Exemplo de Implementação: Lista Simplesmente Encadeada em C

As listas simplesmente encadeadas são estruturas de dados dinâmicas que permitem a inserção e remoção eficiente de elementos em qualquer posição, embora esta implementação foque em operações no início da lista e busca linear. Cada elemento (nó) contém o dado e um ponteiro para o próximo nó na sequência.

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

typedef struct Node{ int x; struct Node* next; } Node;

void insert_front(Node **head,int x){
    Node* n=(Node*)malloc(sizeof(Node)); n->x=x; n->next=*head; *head=n;
}
Node* find(Node* h,int x){ for(;h;h=h->next) if(h->x==x) return h; return NULL; }
bool remove_first(Node **h,int x){
    Node *p=*h,*prev=NULL;
    while(p && p->x!=x){ prev=p; p=p->next; }
    if(!p) return false;
    if(prev) prev->next=p->next; else *h=p->next;
    free(p); return true;
}
void print(Node* h){ for(;h;h=h->next) printf("%d ", h->x); puts(""); }

int main(){
    Node* head=NULL;
    insert_front(&head,3); insert_front(&head,5); insert_front(&head,7);
    print(head);
    printf("find 5? %s\n", find(head,5)?"sim":"nao");
    remove_first(&head,5); print(head);
    return 0;
}
```

Operações Fundamentais da Lista Encadeada

1	Estrutura Node Define o nó básico da lista, contendo um valor inteiro (x) e um ponteiro (next) para o próximo nó na sequência. O último nó aponta para NULL.
2	insert_front (Inserir no Início) Cria um novo nó e o adiciona no início da lista. Este método é eficiente (complexidade O(1)) pois apenas manipula ponteiros sem precisar percorrer a lista.
3	find (Buscar Valor) Percorre a lista do início ao fim para encontrar a primeira ocorrência de um valor específico. Retorna um ponteiro para o nó se encontrado, ou NULL caso contrário.
4	remove_first (Remover Primeira Ocorrência) Localiza e remove a primeira instância de um valor na lista. Gerencia corretamente o encadeamento dos nós e libera a memória do nó removido.
5	print (Imprimir Lista) Percorre a lista e imprime os valores de cada nó. Útil para depuração e para visualizar o estado atual da lista.
6	Exemplo em main Demonstra a sequência de operações: inserções, impressão, busca de um elemento e remoção, mostrando como a lista se comporta e se adapta dinamicamente.

Pilhas: Estrutura LIFO

As pilhas implementam o princípio Last In, First Out (LIFO), onde o último elemento inserido é sempre o primeiro a ser removido. Esta característica torna as pilhas ideais para gerenciar operações que seguem uma ordem inversa de processamento.

1

Push (Empilhar)

Inserir um novo elemento no topo da pilha, tornando-o o elemento mais acessível da estrutura.

2

Pop (Desempilhar)

Remove e retorna o elemento do topo, expondo o elemento anteriormente inserido.

3

Top (Consultar Topo)

Permite visualizar o elemento do topo sem removê-lo da estrutura.



Aplicações Práticas

As pilhas são fundamentais em diversas áreas da computação, desde o gerenciamento básico de chamadas de função até algoritmos complexos de análise sintática e controle de fluxo de execução.

Visualização: Funcionamento de Pilhas

Para compreender melhor o comportamento LIFO, observe como os elementos são organizados e acessados em uma pilha típica.



Base da Pilha

[20]

Primeiro elemento inserido, permanece na base até que todos os elementos superiores sejam removidos.

Elementos Intermediários

[55] [4] [12]

Elementos inseridos em sequência, cada um se tornando temporariamente o topo antes da próxima inserção.

Topo da Pilha

[89]

Último elemento inserido, primeiro a ser removido. Único ponto de acesso para operações push e pop.

Princípio Fundamental: Inserção e remoção ocorrem exclusivamente pelo topo, garantindo a ordem LIFO e preservando a integridade da estrutura.

Exemplo de Implementação: Pilha em C

Para ilustrar os conceitos de pilha (LIFO) de forma prática, apresentamos uma implementação básica em C utilizando um array de tamanho fixo. Este exemplo demonstra as operações fundamentais de empilhar, desempilhar, consultar o topo e verificar o estado da pilha.

```
#include <stdio.h>
#include <stdbool.h>
#define MAX 100

typedef struct { int v[MAX]; int topo; } Stack;

void init(Stack *s){ s->topo = -1; }
bool empty(Stack *s){ return s->topo == -1; }
bool full(Stack *s){ return s->topo == MAX-1; }

bool push(Stack *s, int x){
    if(full(s)) return false;
    s->v[++s->topo] = x; return true;
}
bool pop(Stack *s, int *out){
    if(empty(s)) return false;
    *out = s->v[s->topo--]; return true;
}
bool top(Stack *s, int *out){
    if(empty(s)) return false;
    *out = s->v[s->topo]; return true;
}

int main(){
    Stack s; init(&s);
    push(&s, 10); push(&s, 20);
    int x; top(&s, &x); printf("Topo=%d\n", x);
    while(pop(&s,&x)) printf("pop %d\n", x);
    return 0;
}
```

Funcionalidades Chave

1

init e Verificações

init inicializa a pilha, e as funções empty e full garantem que as operações ocorram de forma segura, evitando erros como empilhar em pilha cheia ou desempilhar de pilha vazia.

2

push (Empilhar)

Adiciona um novo elemento ao topo da pilha. A variável topo é incrementada antes da atribuição para apontar para a próxima posição disponível.

3

pop (Desempilhar)

Remove o elemento do topo da pilha e o retorna. A variável topo é decrementada após a remoção, liberando a posição para futuras inserções.

4

top (Consultar Topo)

Permite visualizar o elemento que está no topo da pilha sem removê-lo, sendo útil para verificar o próximo item a ser processado.

O bloco main exemplifica a sequência de operações, empilhando os valores 10 e 20, consultando o topo e, em seguida, desempilhando os elementos na ordem inversa de inserção, confirmando o princípio LIFO.



Filas: Estrutura FIFO

As filas seguem o princípio First In, First Out (FIFO), simulando comportamentos de espera do mundo real onde o primeiro a chegar é o primeiro a ser atendido. Esta característica torna as filas essenciais para modelar sistemas de atendimento e processamento sequencial.



Enqueue

Operação de inserção que adiciona novos elementos sempre no final da fila, preservando a ordem de chegada.



Dequeue

Operação de remoção que retira elementos sempre do início da fila, respeitando a ordem cronológica de inserção.

Aplicações no Mundo Real

Sistemas Operacionais

Gerenciamento de processos aguardando execução pelo processador.

Impressão

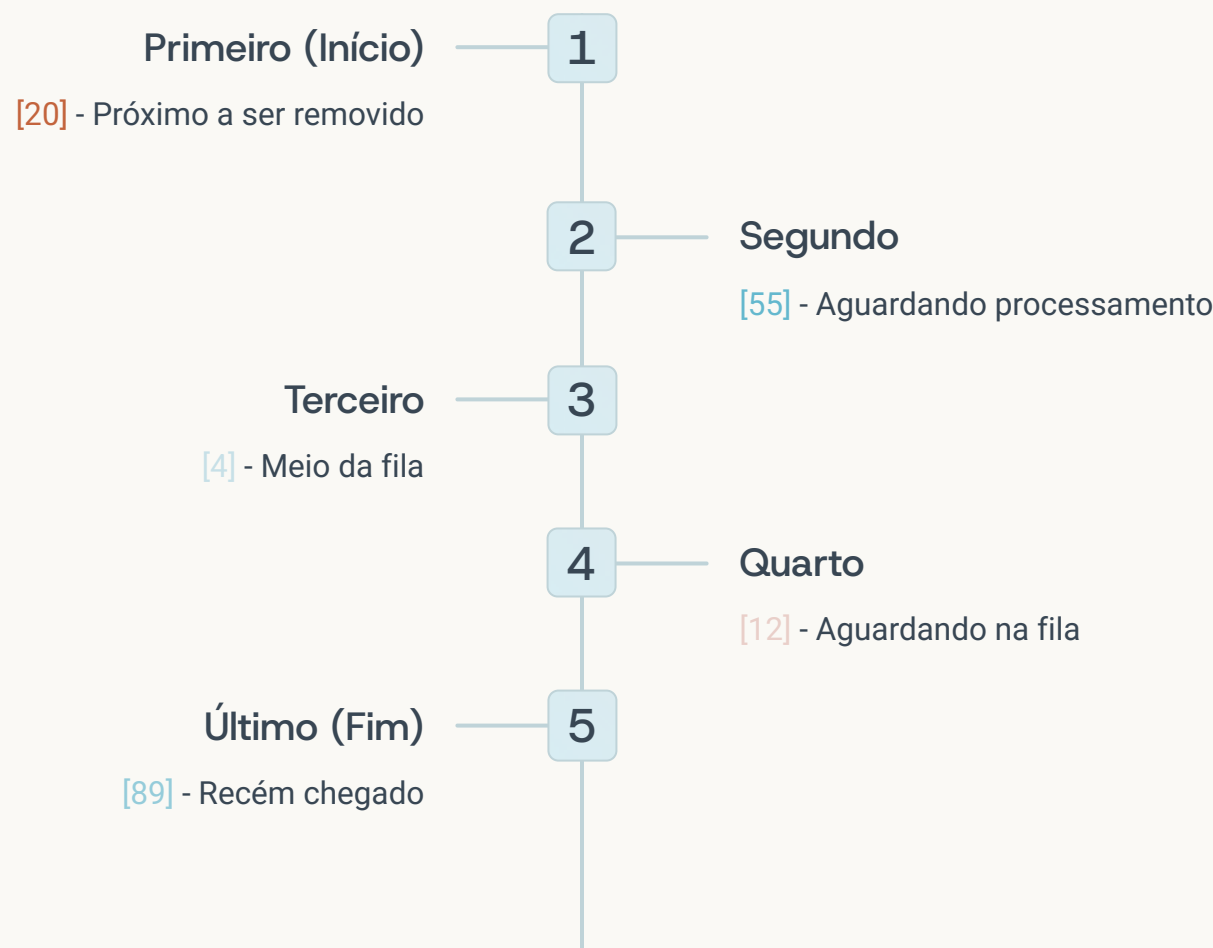
Fila de documentos esperando para serem impressos em ordem.

Simulações

Modelagem de tráfego, atendimento bancário e sistemas de espera.

Visualização: Funcionamento de Filas

A estrutura FIFO das filas pode ser visualizada como uma linha de espera, onde novos elementos chegam pelo final e são atendidos pelo início.



✓ **Fluxo Contínuo:** Entradas no fim e saídas pelo início criam um fluxo contínuo que preserva a ordem temporal dos elementos, fundamental para aplicações que exigem processamento sequencial.

Síntese: Pilhas vs Filas

Compreender as diferenças fundamentais entre pilhas e filas é essencial para escolher a estrutura adequada em diferentes contextos algorítmicos e aplicações práticas.

Estrutura	Ordem de Acesso	Operações Principais	Aplicações Típicas
Pilha	LIFO	push, pop, top	Chamadas de função, expressões aritméticas, operações desfazer
Fila	FIFO	enqueue, dequeue	Fila de impressão, processos do SO, simulações de atendimento

Importância no Contexto Acadêmico

Este conteúdo integra a **Unidade 03 do plano de ensino** de INF0286, fornecendo a base conceitual necessária para compreender estruturas mais complexas como árvores, grafos e algoritmos avançados de pesquisa e ordenação.

Base Conceitual

Fundamento para estruturas de dados mais complexas e algoritmos avançados.

Aplicações Práticas

Uso direto em sistemas reais e computação científica moderna.

Exemplo de Implementação: Fila Circular em C

Para demonstrar o funcionamento de uma Fila Circular (FIFO) na prática, apresentamos uma implementação em C utilizando um vetor de tamanho fixo. Esta abordagem eficiente reusa o espaço em vez de deslocar elementos, crucial para otimização de performance.

```
#include <stdio.h>
#include <stdbool.h>
#include <string.h>
#define N 5 // Capacidade máxima da fila

typedef struct {
    char q[N][32]; // Vetor para armazenar strings de até 31 caracteres + null
    int in, out, cnt; // 'in' aponta para a próxima posição livre, 'out' para o primeiro elemento, 'cnt' para a contagem de elementos
} Queue;

void init(Queue *Q){
    Q->in = Q->out = Q->cnt = 0; // Inicializa todos os ponteiros e contador
}

bool full(Queue *Q){
    return Q->cnt == N; // A fila está cheia se o número de elementos for igual à capacidade máxima
}

bool empty(Queue *Q){
    return Q->cnt == 0; // A fila está vazia se não houver elementos
}

bool enqueue(Queue *Q, const char *s){
    if(full(Q)) return false; // Não é possível enfileirar se a fila estiver cheia
    strncpy(Q->q[Q->in], s, 31); // Copia a string para a posição 'in'
    Q->q[Q->in][31]='\0'; // Garante terminação nula
    Q->in = (Q->in + 1) % N; // Move 'in' para a próxima posição circularmente
    Q->cnt++; // Incrementa o contador de elementos
    return true;
}

bool dequeue(Queue *Q, char *out){
    if(empty(Q)) return false; // Não é possível desenfileirar se a fila estiver vazia
    strcpy(out, Q->q[Q->out]); // Copia o elemento da posição 'out'
    Q->out = (Q->out + 1) % N; // Move 'out' para a próxima posição circularmente
    Q->cnt--; // Decrementa o contador de elementos
    return true;
}

bool front(Queue *Q, char *out){ // Função adicionada para consultar o primeiro elemento sem removê-lo
    if(empty(Q)) return false;
    strcpy(out, Q->q[Q->out]);
    return true;
}

int main(){
    Queue Q; init(&Q); char buf[32];

    printf("Enfileirando A, B, C:\n");
    enqueue(&Q,"A");
    enqueue(&Q,"B");
    enqueue(&Q,"C");

    if(front(&Q, buf)) printf("Primeiro elemento: %s\n", buf); // Demonstração do front

    printf("Desenfileirando todos os elementos:\n");
    while(dequeue(&Q,buf)) {
        puts(buf); // Imprime os elementos na ordem de entrada (FIFO)
    }

    // Demonstração de enfileirar e desenfileirar novamente para mostrar circularidade
    printf("\nEnfileirando D, E, F (Fila cheia depois de E):\n");
    enqueue(&Q,"D");
    enqueue(&Q,"E");
    if (!enqueue(&Q, "F")) printf("Fila cheia, F não pode ser enfileirado.\n");

    printf("Desenfileirando o restante:\n");
    while(dequeue(&Q,buf)) {
        puts(buf);
    }

    return 0;
}
```

Operações Fundamentais da Fila Circular

1	Estrutura da Fila A estrutura <code>Queue</code> armazena os elementos em um vetor <code>q</code>, com <code>in</code>, <code>out</code> e <code>cnt</code> controlando a inserção, remoção e o número atual de elementos, respectivamente.
2	<code>init</code>, <code>empty</code> e <code>full</code> <code>init</code> prepara a fila. As funções <code>empty</code> e <code>full</code> verificam o estado da fila, prevenindo operações inválidas e garantindo a integridade dos dados.
3	<code>enqueue</code> (Enfileirar) Adiciona um elemento ao final da fila. Utiliza o operador módulo (<code>%N</code>) para fazer o ponteiro <code>in</code> avançar circularmente no vetor, reusando o espaço.
4	<code>dequeue</code> (Desenfileirar) Remove e retorna o elemento do início da fila. O ponteiro <code>out</code> avança circularmente, mantendo a ordem FIFO dos elementos.
5	<code>front</code> (Consultar Início) Permite visualizar o elemento que está no início da fila sem removê-lo, essencial para verificar o próximo item a ser processado.
6	Exemplo em <code>main</code> A função <code>main</code> demonstra a sequência completa: enfileiramento, consulta do início, desenfileiramento completo, e um cenário onde a fila fica cheia, ilustrando a robustez da implementação.

Estruturas Estáticas vs. Dinâmicas

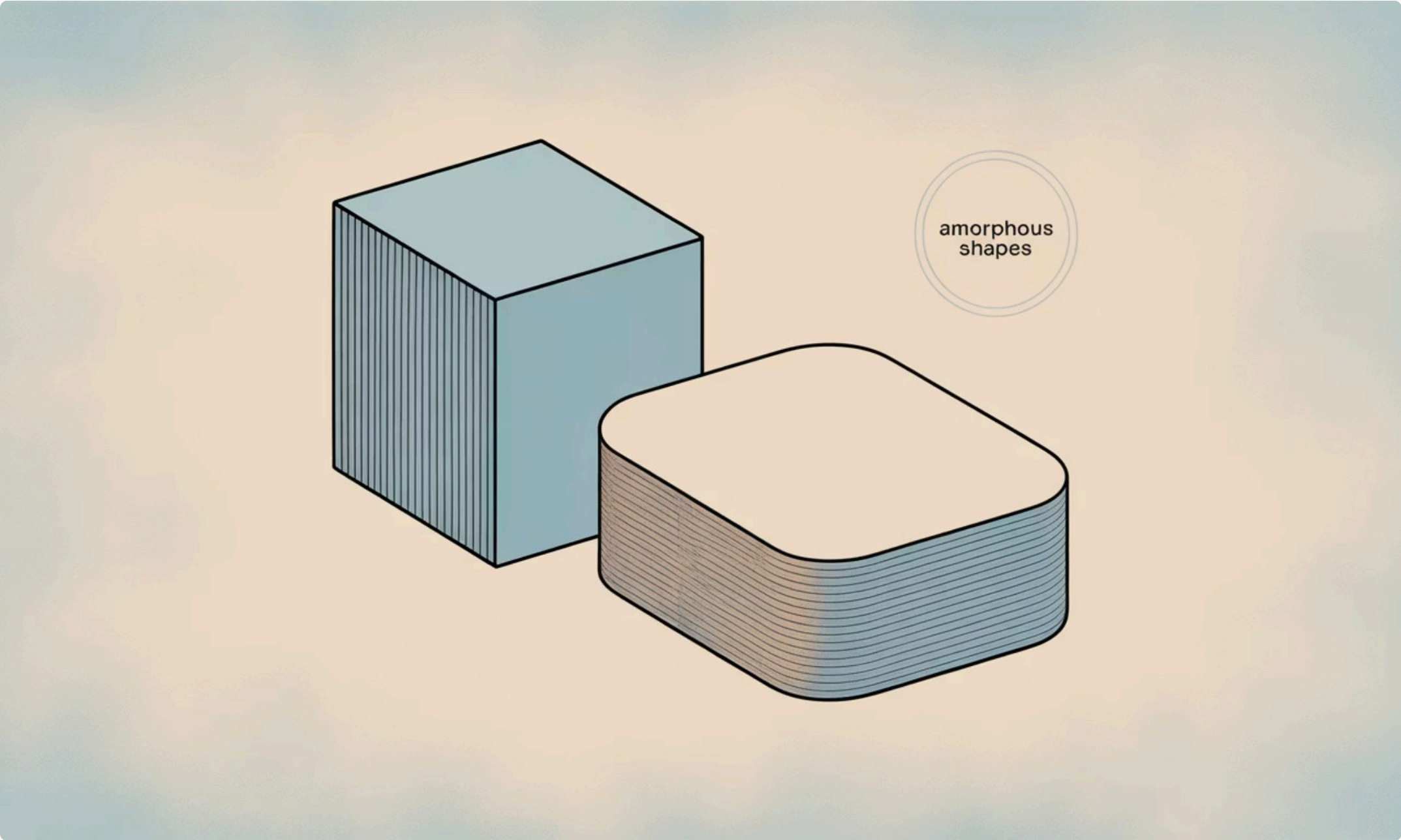
A escolha entre estruturas de dados estáticas e dinâmicas é um aspecto crucial no desenvolvimento de software, impactando diretamente o desempenho, a flexibilidade e a eficiência no uso da memória.

Estruturas Estáticas

- **Tamanho Fixo:** Definido em tempo de compilação e imutável durante a execução.
- **Baseado em Vetores:** Usam arrays ou vetores como base para armazenamento.
- **Acesso Direto:** Permite acesso rápido ($O(1)$) a qualquer posição de memória.
- **Limitações:** Risco de desperdício de memória ou de falta de espaço (overflow) se o tamanho não for precisamente previsto.

Estruturas Dinâmicas

- **Tamanho Flexível:** Ajustável em tempo de execução, crescendo ou diminuindo conforme a necessidade.
- **Baseado em Ponteiros:** Tipicamente implementadas com ponteiros e listas encadeadas.
- **Flexibilidade:** Otimiza o uso da memória, alocando recursos apenas quando necessário.
- **Limitações:** Acesso sequencial (não direto), podendo ser mais lento para localizar elementos específicos.



i A compreensão clara dessas diferenças é fundamental para projetar soluções eficientes, evitando problemas comuns como fragmentação de memória ou gargalos de desempenho em sistemas complexos.

Exemplos de Estruturas Estáticas

1 Pilha com Vetor Fixo

Implementada utilizando um array de tamanho pré-definido, como `int pilha[100]`, onde a capacidade máxima é estabelecida em tempo de compilação.

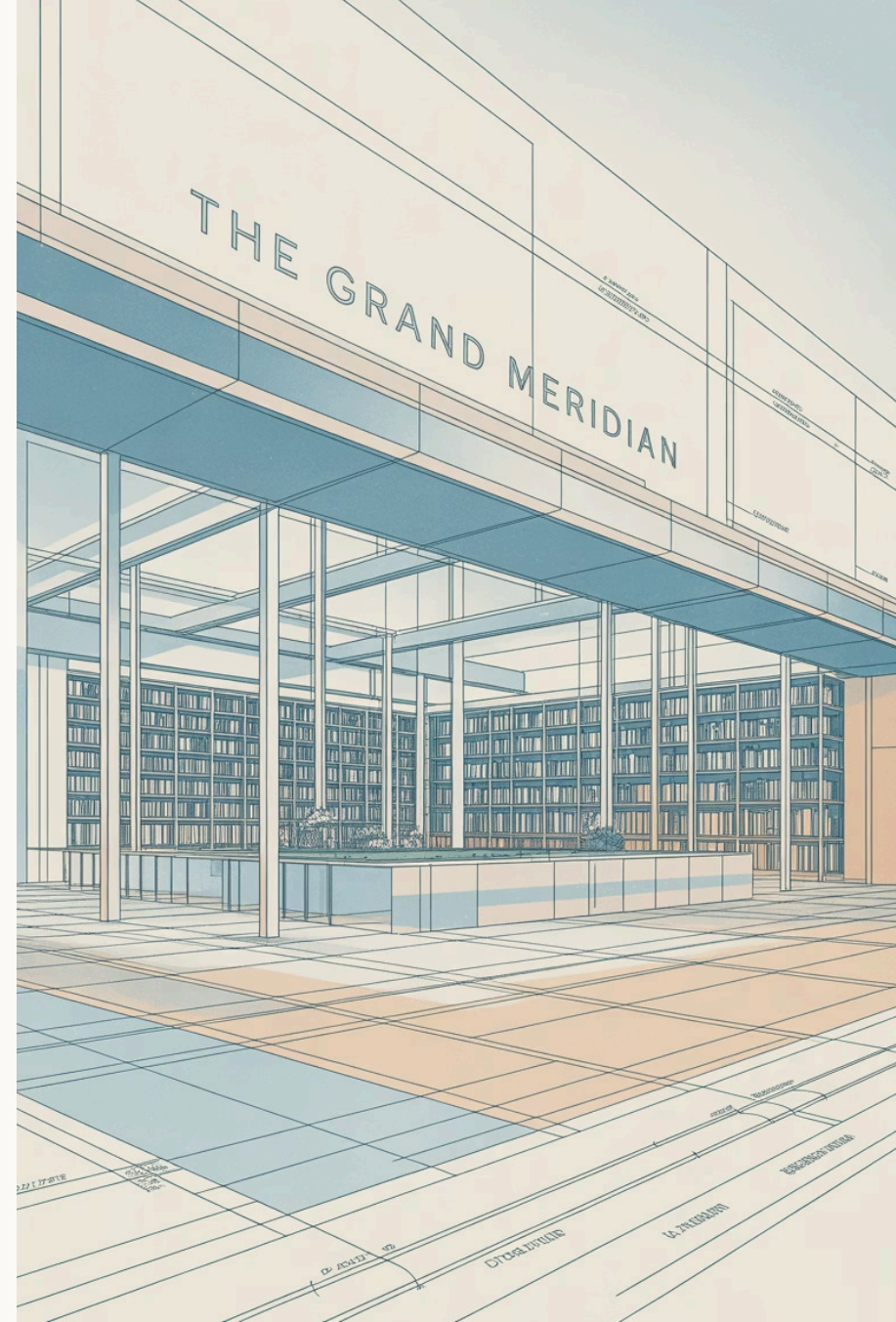
2 Fila Circular em Vetor

Uma implementação eficiente de fila que reusa o espaço de um array fixo, utilizando ponteiros de início e fim que se movem circularmente.

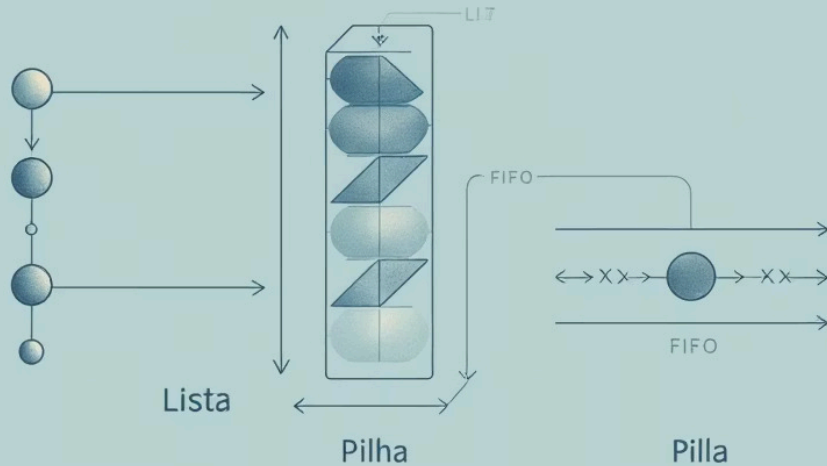
3 Lista Sequencial em Array

Representa uma lista de elementos contíguos na memória, implementada sobre um array, permitindo acesso direto aos elementos por índice.

Essas estruturas são ideais quando o número máximo de elementos é conhecido previamente, otimizando o uso da memória e o acesso direto, mas sem flexibilidade para crescimento.



Exemplos de Estruturas Dinâmicas



Lista Simplesmente Encadeada

Cada nó armazena um dado e um ponteiro para o próximo nó. Ideal para inserções e remoções rápidas em qualquer posição, mas a navegação é unidirecional.

Lista Duplamente Encadeada

Nós possuem ponteiros para o anterior e para o próximo, permitindo navegação eficiente em ambas as direções. Mais complexa, mas oferece maior flexibilidade na manipulação.

Pilha e Fila Dinâmicas

Implementações que utilizam listas encadeadas, onde o tamanho não é fixo. Podem crescer ou diminuir conforme a necessidade, adaptando-se ao volume de dados em tempo real.

A grande vantagem dessas estruturas é a capacidade de ajustar seu tamanho dinamicamente em tempo de execução, otimizando o uso da memória e oferecendo flexibilidade inigualável em aplicações com volume de dados variável.